

2026학년도 2학기 언어데이터과학

제3강 Linux 기초

박수민

서울대학교 인문대학 언어학과

2026년 9월 9일 수요일

지난 시간에 배운 것

- 1 버전 관리와 git의 세 영역
- 2 add → commit → push
- 3 GitHub Codespaces 실습 환경

오늘의 목표

- 1 명령줄에서 파일 시스템을 탐색할 수 있다.
- 2 큰 텍스트 파일을 열지 않고 크기·구조·내용을 확인할 수 있다.
- 3 명령어를 파이프로 이어 붙여 원하는 부분만 잘라낼 수 있다.
- 4 grep과 정규표현식으로 단어를 찾고 세어 볼 수 있다.

사전 준비: 지난 시간에 만든 codespace 다시 열기

- 1 `https://github.com`에서 자신의 LDS2026 저장소에 들어간다.
- 2 [Code] → [Codespaces] 탭에서 **이미 있는 codespace 이름**을 클릭한다.
- 3 중지되어 있으면 다시 켜지는 데 30초쯤 걸린다. 터미널이 열리면 준비 완료.
새로 만들지 않는다. 2강에서 만든 그 환경에 지난번 작업이 그대로 남아 있다.

목록에 아무것도 없다면

지웠거나 아직 만들지 않은 것이다. [Create codespace on main]으로 새로 만든다.

오늘 명령을 실행하는 곳은 내 컴퓨터가 아니라 **클라우드의 리눅스**이다.

1 명령줄 인터페이스

왜 명령줄인가
터미널 읽는 법

2 파일 시스템 탐색

3 텍스트 파일 다루기

4 검색과 빈도

5 마무리

익숙한 풍경

말뭉치 파일을 받은 날

- 1 700 MB짜리 텍스트 파일을 내려받는다.
- 2 더블클릭한다.
- 3 편집기가 멈춘다.
- 4 강제 종료한다.

알고 싶었던 것

- 이 파일은 몇 줄인가?
- 안에 무엇이 들어 있는가?
- 내가 찾는 단어가 있기는 한가?
- 필요한 부분만 어떻게 꺼내는가?

오늘의 방법

파일을 **열지 않고** 알아본다.

두 가지 방식

	GUI (그래픽)	CLI (명령줄)
조작 방법	마우스로 누른다	글자로 적는다
배우기	쉽다	처음에는 어렵다
100번 반복	100번 누른다	한 줄로 끝낸다
작업 기록	남지 않는다	명령 그 자체가 기록이다
큰 파일	열다가 멈춘다	열지 않고 살펴본다
원격 접속	화면 전체를 주고받아야 한다	글자만 주고받으면 된다

둘 중 하나를 고르는 것이 아니다

오늘도 편집은 VS Code(GUI)로, 데이터 처리는 터미널(CLI)로 한다.

명령줄과 재현 가능한 연구

마우스로 한 작업

“엑셀에서 정렬하고, 중복을 지우고, 몇 개는 손으로 고쳤습니다.”

- 6개월 뒤에 같은 결과를 다시 만들 수 없다.
- 남에게 설명할 수도 없다.

명령줄로 한 작업

- 내가 한 일이 한 줄의 글로 남는다.
- 그 줄을 저장소에 커밋하면 방법 자체가 기록된다.
- 다른 사람이 같은 줄을 실행하면 같은 결과가 나온다.

이 수업에서 명령줄을 쓰는 진짜 이유는 빠르기 때문이 아니라 **남기 때문이다.**

프롬프트

터미널을 열면 보이는 것

```
@your-username /workspaces/LDS2026 (main) $
```

읽는 법

@... 지금 로그인한 사용자

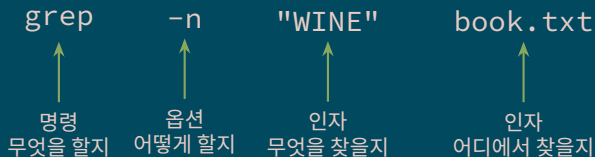
/workspaces/... 지금 내가 있는 위치

(main) 지금 있는 git 브랜치

\$ “명령을 기다린다”는 표시

\$는 명령의 일부가 아니다. **입력하지 않는다.**

명령의 구조



규칙

- 항목은 **빈칸**으로 구분한다.
- 옵션은 붙임표로 시작한다. (-n, --version)
- 대문자와 소문자는 다른 글자다. (-n과 -N)

슬라이드 읽는 법

터미널 입력

이 상자에 있는 것은 터미널에 **그대로 입력**하는 명령이다.

실행 결과

이 상자에 있는 것은 명령을 실행하면 **나오는 출력**이다. 입력하지 않는다.

- 날짜·사용자 이름·파일 크기는 사람마다 다를 수 있다.

막혔을 때

오류 메시지를 끝까지 읽는다. 오타(대소문자, 빈칸, 따옴표)가 원인인 경우가 대부분이다.

1 명령줄 인터페이스

2 파일 시스템 탐색
위치와 목록
경로와 이동

3 텍스트 파일 다루기

4 검색과 빈도

5 마무리

파일 시스템은 나무다

```
/workspaces/LDS2026 — { README.md  
                        — notes/ — 02-20260907.md  
                        — data/ — apicius/ — book.txt
```

용어

디렉터리(directory) 폴더의 다른 이름. 안에 파일과 다른 디렉터를 담는다.

경로(path) 나무의 뿌리에서 목적지까지 가는 길. 구분자는 /이다.

pwd: 나는 어디에 있는가

터미널 입력

```
pwd
```

실행 결과

```
/workspaces/LDS2026
```

읽는 법

pwd는 print working directory의 줄임말이다.

- 명령은 언제나 **현재 위치**를 기준으로 실행된다.
- “파일이 없다”는 오류의 절반은 위치를 착각해서 생긴다.

ls: 여기에 무엇이 있는가

터미널 입력

```
ls  
ls -l
```

실행 결과

```
README.md  notes  
  
total 8  
-rw-r--r-- 1 codespace codespace 118 Sep  9 09:02 README.md  
drwxr-xr-x 2 codespace codespace 4096 Sep  9 09:02 notes
```

맨 앞 글자가 d이면 디렉터리, -이면 파일이다.

ls 옵션

명령	하는 일
ls	이름만 나열한다
ls -l	크기·수정 시각·권한까지 보여준다 (long)
ls -a	숨김 파일도 보여준다 (all) — .git, .gitignore
ls -lh	크기를 읽기 쉬운 단위로 보여준다 (human)
ls -lt	최근에 고친 것부터 보여준다 (time)
ls notes	notes 디렉터리의 내용을 보여준다

옵션은 붙여 쓸 수 있다

ls -l -a -h와 ls -lah는 같은 명령이다.

절대경로와 상대경로

절대경로

뿌리(/)에서부터 적는 길

- /workspaces/LDS2026/notes
- 어디에서 실행하든 같은 곳

상대경로

지금 위치에서부터 적는 길

- notes, data/apicius
- 현재 위치가 바뀌면 가리키는 곳도 바뀐다

약속된 이름

- 현재 디렉터리 (git add .의 그 점)
- 한 단계 위(부모) 디렉터리
- ~ 내 홈 디렉터리

cd: 이동하기

터미널 입력

```
cd notes      # 아래로 한 단계
pwd
cd ..         # 위로 한 단계
pwd
```

실행 결과

```
/workspaces/LDS2026/notes
/workspaces/LDS2026
```

알아 두면 좋은 것

cd만 치면 홈으로, cd -는 직전 위치로 간다. **Tab**은 경로를 자동 완성한다.

확인 질문

경로 읽기

현재 위치가 `/workspaces/LDS2026/data/apicius`일 때,

- 1 `cd ..`를 실행하면 어디로 가는가?
- 2 `cd ../../`를 실행하면 어디로 가는가?
- 3 `ls ../../`는 무엇을 보여주는가?
- 4 `README.md`를 보려면 어떤 경로를 적어야 하는가?

1 명령줄 인터페이스

2 파일 시스템 탐색

3 **텍스트 파일 다루기**
데이터 내려받기
들여다보기
잘라내어 저장하기

4 검색과 빈도

5 마무리

오늘의 문제

질문

고대 로마 사람들은 무엇으로 요리했을까?

자료

문헌 De re coquinaria (Cookery and Dining in Imperial Rome)

저자 Marcus Gavius Apicius (기원후 1세기)

내용 10권, 약 500개의 레시피. 현존하는 가장 오래된 요리책이다.

오늘의 목표

이 책에서 가장 자주 등장하는 식재료를 **명령어만으로** 찾아낸다.

데이터 출처

The screenshot shows the Project Gutenberg website interface. At the top, there's a navigation bar with a search box, a 'Go!' button, and links for 'PayPal' and 'Donate'. Below this is a menu with 'About', 'Frequently Downloaded', 'Main Categories', 'Reading Lists', and 'Search Options'. The main content area displays the title 'Cookery and Dining in Imperial Rome by Apicius' under the Project Gutenberg logo. It includes a brief description of the book as a collection of Roman recipes from the 5th century AD. A 'Read online now' button is present. Below this, a 'Download for free' section offers various formats: EPUB3 (recommended, 2.2 MB), Kindle (via Send-to-Kindle), Kobo/Nook (via USB transfer), and other formats like Plain Text (704 kB) and EPUB (older e-reader). Social media links and a QR code are also visible on the left side of the book's description.

출처 Project Gutenberg <https://www.gutenberg.org/ebooks/29728>

파일 형식 고르기

같은 책, 여러 형식

EPUB/HTML 서식과 그림이 함께 들어 있다. 보기에 좋다.

Plain Text 글자만 들어 있다. 분석하기에 좋다.

원시 텍스트 (plain text)

서식 정보 없이 문자만 나열된 파일. 어떤 도구로도 열 수 있고, 어떤 명령어로도 처리할 수 있다.

- 이 수업에서 다루는 말뭉치는 대부분 이 형식이거나 그 변형(JSON, XML)이다.

작업 디렉터리 만들기

터미널 입력

```
cd /workspaces/LDS2026  
mkdir -p data/apicius  
cd data/apicius  
pwd
```

실행 결과

```
/workspaces/LDS2026/data/apicius
```

읽는 법

mkdir -p 디렉터를 만든다(make directory). -p는 중간 단계까지 한꺼번에 만들고, 이미 있어도 오류를 내지 않는다.

데이터 내려받기

터미널 입력

```
wget https://www.gutenberg.org/cache/epub/29728/pg29728.txt -O book.txt
```

실행 결과

```
book.txt 100%[=====>] 703.54K 1.42MB/s in 0.5s  
2026-09-09 09:05:11 (1.42 MB/s) - 'book.txt' saved [720428/720428]
```

읽는 법

wget 주소에 있는 파일을 내려받는다
-O book.txt 받은 파일을 book.txt라는 이름으로 저장한다
명령으로 받으면 **그 명령이 곧 출처 기록**이 된다.

제대로 받았는지 확인하기

터미널 입력

```
ls -lh book.txt
```

실행 결과

```
-rw-r--r-- 1 codespace codespace 704K Sep  9 09:05 book.txt
```

크기가 0이라면

내려받기가 실패한 것이다. 주소를 다시 확인하고 명령을 다시 실행한다.

wc: 세어 보기

터미널 입력

```
wc book.txt
```

실행 결과

```
19382  110431  720428 book.txt
```

행 수	단어 수	바이트 수	파일 이름
19382	110431	720428	book.txt

wc는 word count의 줄임말이지만, 실제로는 행·단어·바이트를 함께 센다.

wc 옵션

명령	하는 일
<code>wc -l book.txt</code>	행 수만 (lines)
<code>wc -w book.txt</code>	단어 수만 (words)
<code>wc -c book.txt</code>	바이트 수만 (characters)
<code>wc -m book.txt</code>	문자 수만

-c와 -m이 다른 이유

이 파일은 영문자가 대부분이라 두 값이 거의 같다. 그러나 한국어 텍스트에서는 크게 달라진다. “가”는 1글자이지만 3바이트이다.

- 왜 그런지는 9강(문자 인코딩과 유니코드)에서 다룬다.

head: 앞부분 보기

터미널 입력

```
head book.txt
```

실행 결과

```
The Project Gutenberg eBook of Cookery and Dining in Imperial Rome
```

```
This eBook is for the use of anyone anywhere in the United States and  
most other parts of the world at no cost and with almost no restrictions  
whatsoever. You may copy it, give it away or re-use it under the terms  
of the Project Gutenberg License included with this eBook or online  
at www.gutenberg.org. If you are not located in the United States,  
you will have to check the laws of the country where you are located  
before using this eBook.
```

아무 옵션도 주지 않으면 첫 10행을 보여준다. 레시피는 아직 시작도 하지 않았다.

tail: 뒷부분 보기

터미널 입력

```
tail book.txt
```

실행 결과

```
Most people start at our website which has the main PG search  
facility: www.gutenberg.org.
```

```
This website includes information about Project Gutenberg,  
including how to make donations to the Project Gutenberg Literary  
Archive Foundation, how to help produce our new eBooks, and how to  
subscribe to our email newsletter to hear about new eBooks.
```

끝부분도 요리와는 상관이 없다. 이 파일에는 **본문이 아닌** 것이 섞여 있다.

head/tail 옵션

명령	하는 일
head -n 20 book.txt	첫 20행
head -20 book.txt	첫 20행 (-n을 생략한 형태)
tail -n 20 book.txt	마지막 20행
tail -n +100 book.txt	100번째 행부터 끝까지

-n 100과 -n +100은 다르다

-n 100 끝에서부터 100개

-n +100 앞에서부터 세어 100번째 행부터 끝까지

+를 쓸 때는 -n을 생략할 수 없다. (tail +100은 오류)

cat과 less: 전체 보기

cat

파일 전체를 화면에 쏟아낸다.

```
cat book.txt  
cat -n book.txt
```

- -n: 행 번호를 붙인다
- 19,382행이 순식간에 지나간다

less

한 화면씩 멈추어 가며 보여준다.

```
less book.txt  
cat -n book.txt | less
```

- Space** 다음 화면
- b** 이전 화면
- /단어** 검색
- q** 빠져나오기

터미널이 멈춘 것 같으면 대개 less 안에 있는 것이다. q를 누른다.

|: 명령을 이어 붙이기



파이프(pipe)

앞 명령의 **출력**을 뒤 명령의 **입력**으로 넘긴다. 중간 파일을 만들지 않는다.

터미널 입력: 51-100행 보기

```
head -n 100 book.txt | tail -n 50  
tail -n +51 book.txt | head -n 50
```

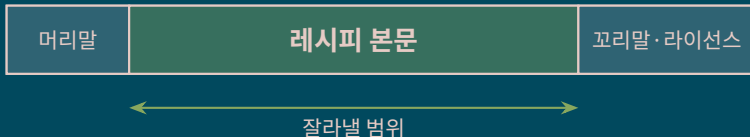
두 줄은 같은 결과를 낸다. 순서를 바꾸어 생각하는 연습을 해 두면 좋다.

확인 질문

파이프 읽기

- 1 `wc -l book.txt`와 `cat book.txt | wc -l`의 결과는 같은가?
- 2 `head -n 100 book.txt | tail -n 50`은 몇 번째 행부터 몇 번째 행까지인가?
- 3 `tail -n 50 book.txt | head -n 100`은 몇 행을 보여주는가?

파일 안에는 본문만 있지 않다



남아 있는 것

구텐베르크 안내문, 표지, 서문, 목차, 라이선스 전문

할 일

- 1 `cat -n`으로 행 번호를 보면서 본문의 시작과 끝을 찾는다.
- 2 그 범위만 잘라 새 파일로 저장한다.

시작 위치 찾기

터미널 입력

```
cat -n book.txt | less
cat -n book.txt | head -n 2942 | tail -n 8
```

실행 결과

```

2935 I
2936
2937
2938 [1] FINE SPICED WINE
2939 _CONDITUM PARADOXUM_
2940
2941 THE COMPOSITION OF [this] EXCELLENT SPICED WINE [is as follows]. INTO
2942 A COPPER BOWL PUT 6 SEXTARII [1] OF HONEY AND 2 SEXTARII OF WINE; HEAT

```

⇒ 첫 번째 레시피는 **2938행**에서 시작한다.

끝 위치 찾기

터미널 입력

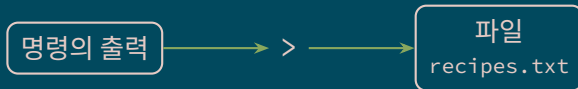
```
cat -n book.txt | tail -n +13696 | head -n 8
```

실행 결과

```
13696 CRUSH IN THE MORTAR PEPPER, CELERY, MINT, AND RUE; MOISTEN WITH
13697 VINEGAR, ADD FIGDATE [wine], HONEY, VINEGAR, BROTH, OIL; LET IT BOIL
13698 LIKEWISE AND SERVE.
13699
13700 [1] This formula evidently is a fragment.
13701
13702
13703 END OF THE SUMMARY OF DISHES [of the Excerpts of Vinidarius]
```

⇒ 마지막 레시피는 **13700행**에서 끝난다.

>: 결과를 파일로 보내기



리다이렉션 (redirection)

화면으로 나갈 출력을 파일로 돌려보낸다. [명령] > [파일 이름]

주의

- > 파일이 이미 있으면 **내용을 지우고** 새로 쓴다
- >> 파일 끝에 이어서 쓴다
- >는 되돌릴 수 없다. 원본 파일 이름을 쓰지 않도록 조심한다.

레시피 본문만 저장하기

터미널 입력

```
head -n 13700 book.txt | tail -n +2938 > recipes.txt  
wc recipes.txt
```

실행 결과

```
10763    57048   367682 recipes.txt
```

읽는 법

- 1 head -n 13700: 처음부터 13700행까지 가져온다.
- 2 tail -n +2938: 그중 2938행부터 끝까지 남기고 파일로 저장한다.
720,428바이트 → 367,682바이트. 절반이 본문이 아니었다.

잠깐, 이것이 데이터 정제다

방금 한 일

연구 대상이 아닌 부분을 걷어내고, 분석할 텍스트만 남긴 파일을 새로 만들었다.

원본을 고치지 않았다는 점이 중요하다

- `book.txt`는 그대로 있다. 잘못 잘랐으면 다시 자르면 된다.
- `recipes.txt`는 명령 한 줄로 언제든지 다시 만들 수 있다.
- 그래서 저장소에는 **데이터가 아니라 명령**을 남긴다.

원칙

원본은 읽기만 한다(read-only). 가공은 언제나 새 파일로 한다.

1 명령줄 인터페이스

2 파일 시스템 탐색

3 텍스트 파일 다루기

4 **검색과 빈도**

grep
정규표현식
빈도 세기

5 마무리

grep: 찾는 단어가 있는 행 출력

터미널 입력

```
grep -n "CABBAGE" recipes.txt
```

실행 결과

```
1460:    CHAP.    IX. YOUNG CABBAGE, SPROUTS, CAULIFLOWER.  
1466:    CHAP.    XV. SOFT CABBAGE.  
1874:[87] YOUNG CABBAGE, SPROUTS [1]  
1994:WRAP THE LEEKS WELL IN CABBAGE LEAVES, HAVING FIRST COOKED THEM AS  
2153:[103] SOFT CABBAGE  
2156:THE CABBAGE IS COOKED WITH POT HERBS IN SODA WATER; PRESS [the water
```

grep은 **행 단위**로 동작한다. 조건에 맞는 행을 통째로 출력한다.

grep 옵션

옵션	하는 일
-n	행 번호를 함께 출력한다
-c	조건에 맞는 행의 개수 를 센다
-i	대소문자를 구별하지 않는다
-v	조건에 맞지 않는 행을 출력한다
-o	맞은 부분만 출력한다 (행 전체가 아니라)
-E	확장 정규표현식을 쓴다 (egrep과 같다)

조합해서 쓴다

```
grep -oE "[A-Z][A-Z][A-Z]+" recipes.txt
```

-c가 세는 것은 무엇인가

터미널 입력

```
grep -c "PEPPER" recipes.txt  
grep -o "PEPPER" recipes.txt | wc -l
```

실행 결과

```
468  
469
```

왜 다른가

-c는 PEPPER가 들어 있는 **행의 수**를, -o | wc -l은 **나타난 횟수**를 센다. 한 행에 두 번 나온 곳이 있는 것이다.

빈도를 보고할 때는 **무엇을 셧는지**를 함께 밝혀야 한다.

정규표현식

정규표현식(regular expression)

글자 그대로가 아니라 **패턴**으로 문자열을 기술하는 표기법.

표기	뜻	예
.	아무 문자 하나	W.NE → WINE, WANE
[A-Z]	대문자 하나	[A-Z][A-Z] → AB
[aeiou]	괄호 안의 글자 하나	모음 하나
*	앞의 것 0번 이상	AB* → A, AB, ABB
+	앞의 것 1번 이상	AB+ → AB, ABB
^	행의 시작	^THE
\$	행의 끝	SERVE\$

+, ?, | 등을 쓰려면 -E 옵션이 필요하다.

패턴으로 단어 뽑아내기

이 책에서 “단어”를 어떻게 정의할까

레시피 본문은 모두 대문자이다. **대문자 3글자 이상의 연쇄**를 한 단어로 본다.

터미널 입력

```
grep -oE "[A-Z][A-Z][A-Z]+" recipes.txt | head -n 4
```

실행 결과

```
FINE  
SPICED  
WINE  
CONDITUM
```

-o는 맞은 부분만 출력한다. 한 행에 한 단어씩 놓여 이제 셀 수 있게 되었다.

단어를 정의한다는 것

이 정의가 버리는 것

- OIL, RUE처럼 3글자인 단어는 남지만 2글자 단어는 버려진다.
- 소문자로 적힌 편집자의 주석은 모두 빠진다. (의도한 것이다)
- PEPPER, PEPPERS, PEPPERCORNS는 서로 다른 단어로 세어진다.

연구자의 결정

무엇을 한 단어로 볼 것인가는 데이터가 아니라 **연구자가 정하는 것이다**. 그 결정을 밝히지 않으면 같은 코퍼스에서 다른 수가 나온다.

- 이 문제는 8강(단어 빈도와 생산성)에서 본격적으로 다룬다.

조건 겹쳐 쓰기

터미널 입력: 세 재료가 모두 나온 행

```
grep -n "PEPPER" recipes.txt | grep "HONEY" | grep "OIL"
```

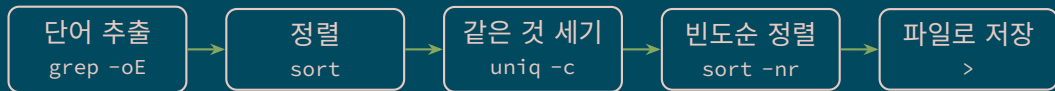
실행 결과

```
643:PEPPER, LOVAGE, CORIANDER, RUE, BROTH, HONEY AND A LITTLE OIL.  
645:ANOTHER WAY: THYME, SATURY, PEPPER, LOVAGE, HONEY, BROTH AND OIL.  
6230:PUT IN A BRAISIÈRE [2] AND BOIL PEPPER, SPICES, HONEY, BROTH; AND HEAT  
6968:WITH OIL, BROTH, REDUCED WINE, WINE, PEPPER, AND HONEY. WHEN DONE  
7015:BRAISE THE TRUFFLES WITH PEPPER, MINT, RUE, HONEY, OIL, AND A LITTLE
```

주의

지금 찾은 것은 **한 행**에 함께 나온 재료이지, 한 **레시피**에 함께 쓰인 재료가 아니다.

파이프라인 설계



생각하는 순서

- 1 세려면 먼저 한 줄에 한 단어씩 놓아야 한다.
- 2 같은 단어끼리 모여 있어야 셀 수 있다.
- 3 세고 나면 많은 것부터 보고 싶다.

sort와 uniq

sort: 줄을 정렬한다

```
grep -oE "[A-Z][A-Z][A-Z]+"  
↪ recipes.txt | sort | head -n 5
```

```
ABLE  
ABOUT  
ABOVE  
ABOVE  
ABOVE
```

uniq -c: 같은 줄을 묶어 센다

```
... | sort | uniq -c | head -n 5
```

```
1 ABLE  
1 ABOUT  
32 ABOVE  
1 ABSINTHIUM  
1 ABSORB
```

반드시 기억할 것

uniq는 **붙어 있는** 중복만 묶는다. 그래서 언제나 sort를 먼저 한다.

빈도순으로 정렬하기

정렬 옵션

- n 수의 크기 순으로 정렬한다 (numeric). 빼면 100이 99보다 앞에 온다.
- r 큰 것부터 내림차순으로 정렬한다 (reverse).

터미널 입력: 전체 파이프라인

```
grep -oE "[A-Z][A-Z][A-Z]+" recipes.txt | sort | uniq -c | sort -nr >  
↪ word-freq.txt  
wc -l word-freq.txt
```

실행 결과: 서로 다른 단어의 수

```
2434 word-freq.txt
```

고빈도 30개 단어

터미널 입력

```
head -n 30 word-freq.txt
```

```
1265 AND
1009 THE
 726 WITH
 467 PEPPER
 400 BROTH
 370 WINE
 338 OIL
 245 SAUCE
 226 HONEY
 212 ADD
```

```
197 LITTLE
192 VINEGAR
184 LOVAGE
175 SERVE
169 THIS
164 WHEN
164 FOR
158 THEM
149 CRUSH
148 ANOTHER
```

```
140 ALITER
132 PUT
129 CUMIN
123 COOKED
116 DISH
112 SPRINKLE
109 CORIANDER
108 SEED
103 RUE
102 PAN
```

결과 읽기

기능어

AND, THE, WITH, THIS, FOR, THEM

- 어느 영어 텍스트에서나 상위에 온다.
- 이 텍스트의 특징을 알려주지 않는다.

내용어

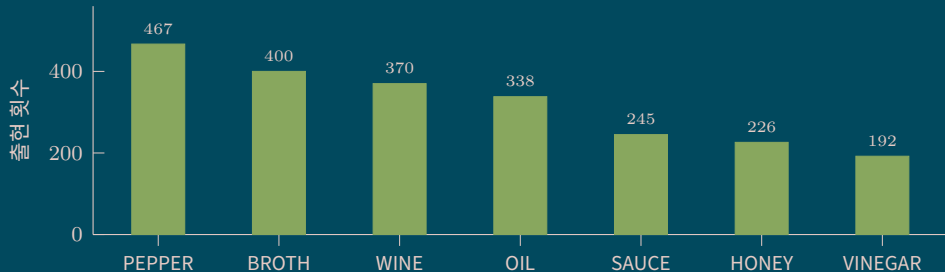
PEPPER, BROTH, WINE, OIL, HONEY

- 이 책이 무엇에 대한 책인지 알려준다.
- ALITER(“달리 하는 법”)는 이 문헌 고유의 형식어이다.

빈도 분석의 기본 구조

고빈도어 목록은 **기능어**와 **내용어**로 갈린다. 무엇을 버리고 무엇을 볼 것인지가 분석의 시작이다.

로마 요리의 재료



읽을 수 있는 것

후추가 압도적이고, 액체 조미료(육수·포도주·기름)가 그 뒤를 잇는다. 꿀과 식초가 함께 상위에 있다는 점은 단맛과 신맛을 같이 쓰는 조리법을 시사한다.

프란과 함께 쓰인 것은?

터미널 입력

```
grep "CUMIN" recipes.txt | grep -oE "[A-Z][A-Z][A-Z]+" | sort | uniq -c |  
↪ sort -nr | head -n 20
```

```
129 CUMIN  
77 PEPPER  
62 AND  
37 LOVAGE  
32 WITH
```

```
29 BROTH  
27 CRUSH  
26 SEED  
26 HONEY  
24 CORIANDER
```

```
22 WINE  
22 RUE  
22 MINT  
18 LITTLE  
16 VINEGAR
```

```
16 LASER  
15 THE  
14 OIL  
13 DRY  
12 ORIGANY
```

앞의 파이프라인에 `grep "CUMIN"` 한 단계만 붙였다.

공기어 목록 읽기

프란(CUMIN)과 함께 나타난 향신료·향신채

스파이스 후추(PEPPER), 고수씨(CORIANDER)

허브 러비지(LOVAGE), 운향(RUE), 박하(MINT), 실피움(LASER), 오레가노(ORIGANY)

이것이 연어(collocation) 분석의 출발점이다

- 어떤 단어가 어떤 단어와 함께 나타나는가?
- 우연히 함께 나타난 것과 정말로 자주 붙어 다니는 것을 어떻게 구별하는가?
- 지금은 “같은 행에 있는 것”으로 근사했을 뿐이다.

1 명령줄 인터페이스

2 파일 시스템 탐색

3 텍스트 파일 다루기

4 검색과 빈도

5 **마무리**

요약

다음 시간 예고

할 일

오늘 쓴 명령

명령	하는 일
<code>pwd</code>	지금 있는 위치를 보여준다
<code>ls -l, ls -a</code>	무엇이 있는지 보여준다
<code>cd</code>	다른 디렉터리로 이동한다
<code>mkdir -p</code>	디렉터리를 만든다
<code>wget -O</code>	파일을 내려받는다
<code>wc</code>	행·단어·바이트를 센다
<code>head, tail</code>	앞부분·뒷부분을 본다
<code>cat -n, less</code>	전체를 본다
<code>grep -n -c -o -E</code>	패턴에 맞는 것을 찾는다
<code>sort, uniq -c</code>	정렬하고 같은 것을 센다
<code> , ></code>	명령을 잇고, 결과를 파일로 보낸다

자주 만나는 오류

메시지 · 증상	대처
No such file or directory command not found	pwd와 ls로 위치와 이름을 확인한다 명령 이름의 철자를 확인한다
tail: cannot open '+100' 화면이 멈춘 것 같다 결과가 끝없이 쏟아진다 빈 파일이 만들어졌다	tail -n +100으로 쓴다 less 안이다. q를 누른다 Ctrl+C로 중단한다 >의 왼쪽 명령이 실패했는지 확인한다

습관

긴 파이프라인은 한 번에 만들지 않는다. 한 단계씩 붙이면서 head로 결과를 확인한다.

이 방법으로 할 수 없는 것

오늘 알아낸 것

책 전체에서 어떤 단어가 몇 번 나왔는지, 어떤 단어들이 같은 **행**에 있었는지

알 수 없는 것

- PEPPER가 나온 곳이 **어떤 요리**인지 모른다.
- 한 **레시피** 안에서 함께 쓰인 재료를 셀 수 없다.
- 레시피가 모두 몇 개인지도 아직 모른다.

원인

grep에게 이 파일은 그저 **10,763개의 행**일 뿐이다. 레시피라는 단위는 사람 눈에만 보인다. → 텍스트에 구조를 부여해야 한다.

확인 질문

정리

- 1 `recipes.txt`의 행 수를 세는 명령은 무엇인가?
- 2 `sort`를 빼고 `uniq -c`만 쓰면 결과가 어떻게 달라지는가?
- 3 `grep -c "WINE"`과 `grep -o "WINE" | wc -l`의 값이 다른 이유는?
- 4 `head -n 100 book.txt > book.txt`를 실행하면 어떤 일이 생기는가?

오늘 배운 것

- 1 명령줄에서 파일 시스템 탐색하기: `pwd`, `ls`, `cd`, `mkdir`
- 2 파일을 열지 않고 살펴보기: `wc`, `head`, `tail`, `less`
- 3 명령 잇기와 결과 저장하기: `|`, `>`
- 4 `grep`과 정규표현식으로 찾고 세기: `sort`, `uniq -c`

다음 시간에 배울 것 (9월 14일 월요일)

- 1 AWK로 텍스트에 구조 부여하기
- 2 요리책 한 권을 레시피 단위로 쪼개기

다음 시간까지 할 일

오늘 한 일을 저장소에 남기기

- 1 notes/03-20260909.md에 오늘 쓴 명령을 붙여 넣는다.
- 2 데이터 원본은 저장소에 넣지 않는다. .gitignore에 data/를 적는다.

터미널 입력

```
cd /workspaces/LDS2026
echo "data/" >> .gitignore
git add .
git commit -m "3강 실습 명령 기록 추가"
git push
```

데이터가 아니라 **데이터를 만든 명령**을 남기는 것이 재현 가능한 연구의 첫걸음이다.